Universidad de la Cuenca del Plata
Autorización Definitiva. Decreto Poder Ejecutivo Nacional N°091/2006

AA 28 - La web del terror accesible

Auditoría de Calidad (ISO 25000), Accesibilidad (WCAG 2.1) y Testing de Software

CARRERA: Ingeniería en Sistemas de Información

MATERIA: Paradigmas y Lenguajes de Programación III

COMISIÓN: “A”

PROFESOR: Ing. Sicardi Dante

ESTUDIANTE: De Olivera, Jesús

Introducción

En este trabajo retomé el proyecto del Desafío de Halloween (la página para votar disfraces con la API REST en PHP y MySQL) y lo revisé con tres enfoques: la accesibilidad según las pautas WCAG 2.1 nivel AA, las pruebas del software y la calidad según la norma ISO/IEC 25010.

El proyecto terminado está en la carpeta AA28-halloween-accesible del repositorio y se ejecuta en XAMPP igual que la versión anterior.

Repositorio: https://github.com/JesusDeOlivera/Paradigmas-III/tree/main/AA28-halloween-accesible

1. Refactorización de accesibilidad

Auditoría inicial

Primero pasé Lighthouse con la categoría Accesibilidad sobre cada página (inicio, ingreso, registro y panel de administración). Después recorrí todo el sitio usando solo el teclado.

[Pegar acá la captura de Lighthouse de la versión anterior]

La puntuación inicial fue de ____ **/ 100**.

Entre la auditoría automática y la prueba con teclado encontré estos problemas:

1. **Atributo ARIA prohibido.** El puesto de cada disfraz en el ranking (🥇, 🥈, #4...) era un span con aria-label. Lighthouse marca esto como error porque un span no tiene rol y el aria-label no se aplica a elementos genéricos.
2. **Contraste insuficiente en el panel de administración.** Las filas de los disfraces que estaban en la papelera tenían opacity: 0.55, y eso bajaba el contraste del texto a 2,3:1. El mínimo es 4,5:1.
3. **El foco se perdía al votar.** Al apretar Enter sobre "Votar", la lista se volvía a cargar desde la API y el botón que tenía el foco desaparecía. El foco volvía al principio de la página y había que recorrer todo otra vez. En el panel pasaba lo mismo al eliminar o restaurar.
4. **No había forma de saltear el menú.** Para llegar a los disfraces había que pasar siempre por todos los enlaces de la cabecera.
5. **Botones con el mismo nombre.** Había seis botones "🎃 Votar" y seis "Editar", y un lector de pantalla no puede saber de qué disfraz es cada uno. Además leía el emoji ("calabaza votar").
6. **Errores de formulario poco claros.** Si faltaba un dato aparecía el mensaje, pero el campo no se marcaba ni recibía el foco.
7. **Scroll horizontal en el celular.** Probando con la vista de celular de Chrome vi que el panel de administración quedaba más ancho que la pantalla (493 px en una pantalla de 400) y había que moverse de costado. Lighthouse no lo marca, pero va contra la pauta 1.4.10 (Reflow) de WCAG 2.1.

Correcciones

Puesto del ranking. Reemplacé el aria-label por un texto oculto visualmente que sí lee el lector de pantalla, y dejé la medalla con aria-hidden para que no se lea:

```
<!-- Antes -->
<span class="puesto" aria-label="Puesto 1">🥇</span>

<!-- Después -->
<span class="puesto">
    <span aria-hidden="true">🥇</span>
    <span class="solo-lector">Puesto 1</span>
</span>
```

Contraste de la papelera. Saqué la opacidad y marqué las filas eliminadas con un fondo rojizo y el nombre tachado. El texto mantiene su color y el contraste vuelve a superar 4,5:1:

```
/* Antes */
.fila-eliminada { opacity: 0.55; }

/* Después */
.fila-eliminada { background-color: rgba(255, 77, 109, 0.06); }
.fila-eliminada .tabla-nombre { text-decoration: line-through; }
```



Figura 1. Fila en la papelera sin opacidad. El foco quedó en "Restaurar" después de eliminar.

Foco visible. En el proyecto no había ningún outline: none, pero reforcé el estilo del foco para que se vea bien sobre el fondo oscuro. Usé un contorno verde de 3 px, que contrasta 13:1 con el fondo, y un halo alrededor de los botones. También agregué scroll-padding-top, porque la cabecera es fija y un elemento que recibía el foco podía quedar tapado debajo de ella:

```
:focus-visible {
    outline: 3px solid var(--pocion);
    outline-offset: 3px;
}

html {
    scroll-padding-top: 96px;
}
```

Saltar al contenido. Agregué en las cuatro páginas un enlace que aparece con el primer Tab y lleva directo al contenido principal:

```
<a href="#contenido" class="saltar-contenido">Saltar al contenido principal</a>
...
<main id="contenido" tabindex="-1">
```



Figura 2. El primer Tab muestra el enlace "Saltar al contenido principal".

Devolver el foco después de cada fetch(). Esta fue la corrección más importante. Después de votar, cuando la lista termina de recargarse, el foco vuelve al botón del mismo disfraz:

```
avisar(datos.mensaje, 'exito');
await cargarDisfraces();   // se vuelve a dibujar la lista

// el botón anterior ya no existe: se enfoca el nuevo botón del mismo disfraz
enfocar(`.tarjeta-disfraz[data-id="${id}"] .boton-votar`);
```

En el panel de administración hice lo mismo. Después de eliminar, el foco pasa a "Restaurar" de esa fila. Después de restaurar o de guardar una edición, vuelve a "Editar".



Figura 3. Después de votar con Enter, el foco sigue en el botón, que ahora dice "Votado".

Nombres claros en los botones. Cada botón tiene un texto oculto con el nombre del disfraz, y el emoji va con aria-hidden. Para el lector de pantalla los botones son "Votar por Calabaza gigante", "Editar Bruja del bosque", etc. En pantalla se siguen viendo igual que antes.

Errores de formulario. Cuando falta un dato o la contraseña es incorrecta, el campo se marca con aria-invalid="true" y recibe el foco. El mensaje se anuncia en una región aria-live:

```
function marcarError(campo) {
    campo.setAttribute('aria-invalid', 'true');
    campo.focus();
}
```



Figura 4. Contraseña incorrecta: el mensaje se anuncia y el foco vuelve al campo.



Figura 5. Formulario enviado vacío: el foco va al primer campo incompleto.

Vista en el celular. El problema era la tabla de disfraces: estiraba la columna de la grilla y, además, el texto oculto de los botones tenía position: absolute y se ubicaba fuera de la pantalla. Con dos reglas la tabla se desplaza dentro de su recuadro y la página entra justa incluso a 320 px de ancho:

```
.admin-grilla > * {
    min-width: 0;
}

.tabla-contenedor {
    position: relative;
    overflow-x: auto;
}
```

Comunicación asíncrona accesible

Todas las respuestas de los fetch() se muestran en regiones con aria-live="polite" y aria-atomic="true": el estado de carga del ranking, los mensajes de los formularios y el aviso flotante ("¡Voto registrado!", "Disfraz restaurado", etc.). Así un lector de pantalla anuncia el resultado sin que el usuario tenga que buscarlo.

```
<p class="estado-carga" id="estado-carga" role="status"
   aria-live="polite" aria-atomic="true">Invocando disfraces…</p>
```

El texto siempre se inyecta con .textContent, nunca con innerHTML, así que el nombre o la descripción de un disfraz no puede ejecutar código aunque tenga etiquetas HTML. Mientras se espera la respuesta, el botón que hizo el pedido queda deshabilitado.

Prueba de teclado

Hice el recorrido completo sin tocar el mouse:

- El primer Tab muestra "Saltar al contenido principal" y con Enter el foco pasa al contenido.
- Con 4 Tab llego a "Ingresar". Ingreso escribiendo el usuario, Tab y la contraseña, y con Enter se envía el formulario.
- Con Tab llego a "Votar por Momia egipcia" y voto con Enter. El foco se queda en el mismo botón, y con la barra espaciadora quito el voto.
- En el panel, con Enter sobre "Editar" el foco pasa al campo nombre. Lo cambio y con Enter se guarda; el foco vuelve a "Editar".
- Con Enter sobre "Eliminar" aparece la confirmación, que también se acepta con Enter, y el foco pasa a "Restaurar".

En todos los pasos el elemento con el foco se ve claramente.

Por último volví a correr Lighthouse sobre el proyecto terminado. Tanto la página de inicio como el panel de administración dieron 100/100:

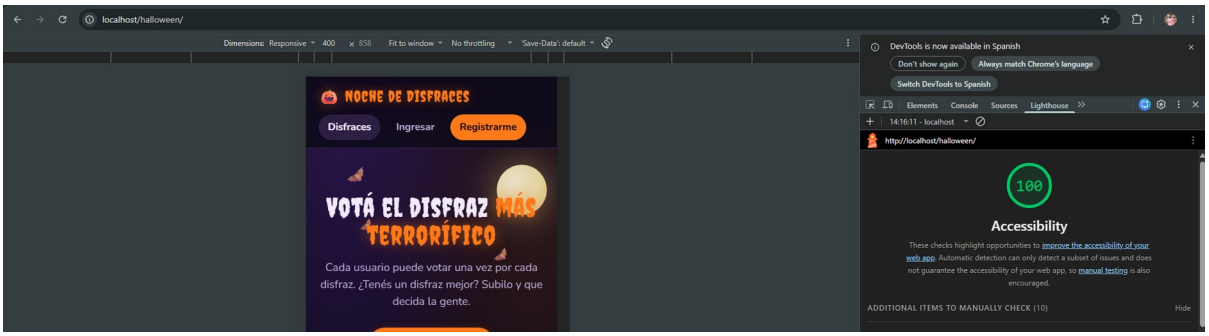


Figura 6. Lighthouse sobre la página de inicio: 100/100 en Accesibilidad.

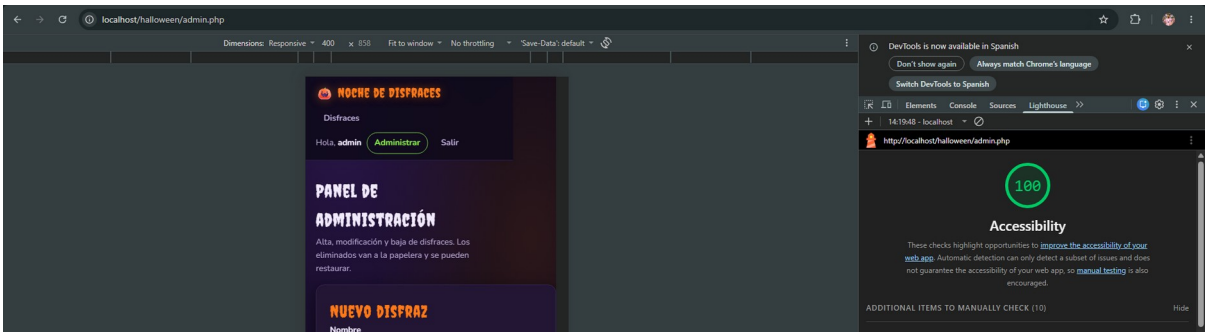


Figura 7. Lighthouse sobre el panel de administración: 100/100 en Accesibilidad.

2. Pruebas automáticas (Testing de Software)

Escribí un script en PHP, tests/test_api.php, que prueba la API real haciendo pedidos HTTP igual que el navegador. Se corre con el PHP que trae XAMPP:

```
C:\xampp\php\php.exe tests\test_api.php
```

Hace 30 pruebas en cinco grupos: listado público, seguridad, registro y votos, CRUD del administrador y tiempo de respuesta. Además de los casos normales, prueba que el sistema rechace lo que tiene que rechazar: un intento de inyección SQL en el login, un voto duplicado, un usuario común que intenta borrar un disfraz, un SVG con un script adentro y un pedido sin la cabecera anti-CSRF. Este fue el resultado:

```
Pruebas de la API — http://localhost/halloween/

1. Listado público (Adecuación funcional)
  [OK]    GET api/disfraces.php responde 200 con JSON
  [OK]    El listado trae disfraces desde MySQL
  [OK]    El listado está ordenado por votos (ranking)
  [OK]    Disfraz inexistente → 404
  [OK]    Id no numérico → 400
  [OK]    La foto se sirve desde el BLOB como imagen

2. Seguridad (OWASP)
  [OK]    Votar sin sesión → 401
  [OK]    Inyección SQL en el login es rechazada → 401
  [OK]    Contraseña incorrecta → 401 con mensaje genérico
  [OK]    Petición sin cabecera anti-CSRF → 403
```

```
3. Registro, login y votos
  [OK]    Registro de 'test_6dbc45' → 201 e inicia sesión
  [OK]    Registrar el mismo usuario otra vez → 409
  [OK]    Registro con datos inválidos → 400
  [OK]    Login con la contraseña correcta (password_verify) → 200
  [OK]    La cookie de sesión es HttpOnly y SameSite
  [OK]    Votar con sesión → 201
  [OK]    El contador de votos sube en 1
  [OK]    Voto duplicado → 409
  [OK]    Quitar el voto → 200 y el contador vuelve al valor anterior
  [OK]    Un usuario común no puede borrar disfraces → 403
  [OK]    Subir un SVG con script es rechazado → 415
  [OK]    Un usuario común no puede abrir admin.php → 403

4. CRUD del administrador
  [OK]    Login del administrador → 200
  [OK]    Crear disfraz con foto → 201
  [OK]    Editar disfraz (PUT) → 200
  [OK]    Enviar a la papelera (borrado lógico) → 200
  [OK]    El disfraz en la papelera ya no es visible para el público → 404
  [OK]    Borrado definitivo → 200 (limpieza de la prueba)
  [OK]    Método no soportado → 405

5. Eficiencia de desempeño
  GET api/disfraces.php × 20: promedio 0.6 ms · p95 0.7 ms · máximo 0.7 ms
  [OK]    El listado responde en menos de 500 ms (p95)

Resultado: 30 de 30 pruebas aprobadas
```

3. Ficha de Autoevaluación de Calidad ISO/IEC 25010

Revisé cada característica en el código y con las pruebas anteriores. La ficha también está en el repositorio, en el archivo CALIDAD-ISO25010.md.

Característica ISO 25010	**Evidencia técnica en el código**	**Estado**
1. Adecuación Funcional	El backend procesa y guarda los datos en MySQL con PDO (`includes/conexion.php`). Los endpoints de `api/` hacen el alta, la baja, la modificación y la consulta de disfraces, usuarios y votos. El voto se guarda en la tabla votos y en el contador del disfraz dentro de una misma transacción.	Cumple
2. Eficiencia de Desempeño	El listado no manda las imágenes, solo su dirección. Las fotos se piden aparte a `api/foto.php` con caché de 24 horas y se comprimen a menos de 60 KB antes de subirlas. El script de pruebas mide 20 pedidos al listado y exige menos de 500 ms.	Cumple
3. Compatibilidad	El frontend (HTML, CSS y JavaScript Vanilla) no tiene PHP mezclado y se comunica con el backend solo por `fetch()` y JSON. La API la puede usar cualquier cliente: el propio script de pruebas la usa sin navegador.	Cumple

Característica ISO 25010	Evidencia técnica en el código	Estado
4. Fiabilidad	Cada endpoint captura `PDOException`, guarda el detalle con `error_log` y responde un 500 genérico. Se usan los códigos 200, 201, 400, 401, 403, 404, 405, 409, 413, 415 y 500 según el caso, y los votos usan transacciones con commit y rollback.	Cumple
5. Seguridad (OWASP)	`password_hash` con BCRYPT al registrarse y `password_verify` al ingresar. Todas las consultas son preparadas (`prepare` + `bindValue`). Además: cookie de sesión HttpOnly y SameSite, cabecera anti-CSRF, validación de imágenes y `textContent` contra XSS. Pendiente: limitar los intentos de login y usar HTTPS.	Cumple
6. Usabilidad / Accesibilidad	Lighthouse marca 100/100 en las cuatro páginas. Todo el sitio se usa con Tab, Enter y Espacio, el foco se ve con `:focus-visible` y vuelve al mismo botón después de cada acción. Los resultados de cada `fetch()` se anuncian con `aria-live="polite"`. Se adapta al celular sin scroll horizontal, incluso a 320 px.	Cumple
7. Mantenibilidad	El código está separado en capas: `includes/` (conexión, sesión y utilidades), `api/` (un archivo por recurso), `js/`, `css/`, `sql/` y `tests/`. Está versionado en GitHub. Pendiente: la cabecera HTML se repite en las cuatro páginas.	Cumple

Característica que exigió más trabajo

La que más trabajo me dio fue Usabilidad / Accesibilidad. Los errores de Lighthouse fueron pocos y rápidos de corregir. El problema más serio apareció recién al probar con el teclado: cada vez que se votaba, la lista se volvía a dibujar con los datos nuevos de la API y el foco se perdía. Para arreglarlo tuve que decidir, para cada operación asíncrona, a qué botón tenía que volver el foco una vez que llegaba la respuesta del servidor.

Mejoras pendientes

- Limitar la cantidad de intentos de inicio de sesión para evitar ataques de fuerza bruta, y usar HTTPS cuando el sitio esté publicado.
- Generar la cabecera y el pie desde un solo archivo, para no repetirlos en las cuatro páginas.
- Automatizar también las pruebas de la interfaz. Hoy la prueba con teclado se hace a mano.

4. Conclusión

Con este trabajo me di cuenta de que un sitio puede funcionar bien con el mouse y aun así ser muy difícil de usar con el teclado o con un lector de pantalla, sobre todo cuando la página se actualiza con fetch() sin recargarse. Lighthouse sirvió para los errores puntuales, pero la prueba real fue usar el sitio solo con el teclado. Las pruebas automáticas y la ficha ISO 25010 también me ayudaron a revisar el proyecto completo y no solo la parte visual.